WQL Studio

A WQL workbench for Workday Extend, browse business objects, run WQL, drill into references, and insert / update / delete records on your custom business objects.

WQL Studio is a “Toad for Workday Extend”: a fast way to explore the data model and work with data through the Workday Query Language (WQL) and the Extend API. This guide walks through connecting a tenant and using each mode.

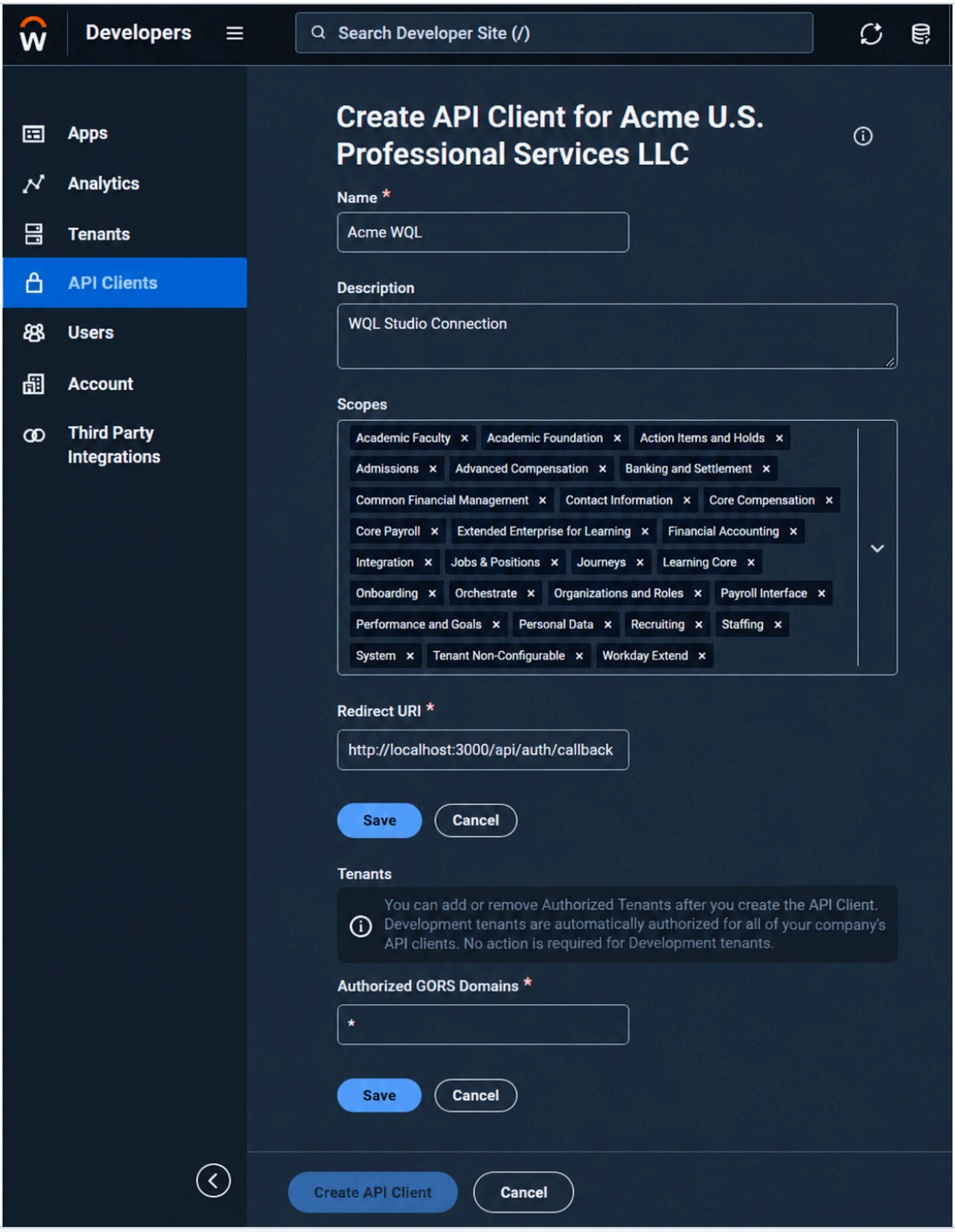
> **Two kinds of work. Select** queries *any* business object (read-only). **Insert / Update / Delete** and **Export/Import** work only on your tenant’s *custom* business objects, the ones whose name carries your `BO Company ID` marker.

Login & setup

WQL Studio signs in to a Workday tenant using OAuth **Authorization Code + PKCE** against an API client registered on that tenant. Your tenant connections are stored locally in your browser; the access token is held server-side and never exposed to the page.

1. Before you start, register an API client

In the Workday **Developer Site**, go to **API Clients** and create one for the tenant you want to query.



Creating the API client on the Workday Developer Site, note the Redirect URI.

Field	What to enter
`Name`	Anything recognisable, e.g. “Acme WQL”.
`Scopes`	The functional areas you intend to query. Include **Workday Extend** so your custom business objects are reachable; add others (Personal Data, Staffing, Core Compensation…) as your use requires. You can only query what the client is scoped for.
`Redirect URI`	Exactly `http://localhost:3000/api/auth/callback`.
`Authorized CORS Domains`	Required by the form. Set per your organisation's security guidance.

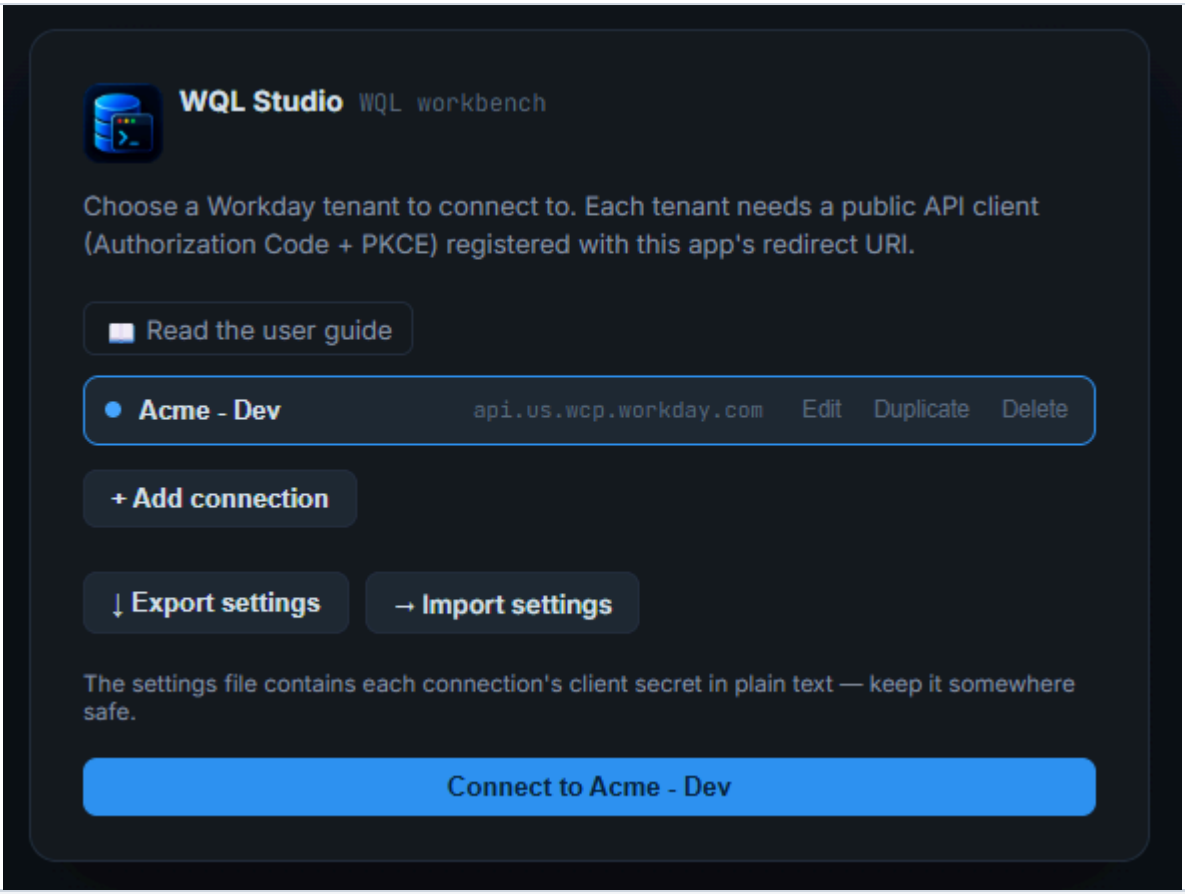
> **The redirect URI must match exactly, including the port.** WQL Studio always runs on `localhost:3000` so this one registered URI works for everyone, on their own machine. If something else is already using port 3000, the app won't be reachable there and login will fail.

After saving, Workday shows the client's **Client ID** and **Client Secret**. Keep them handy, they go straight into the connection form in the next step.

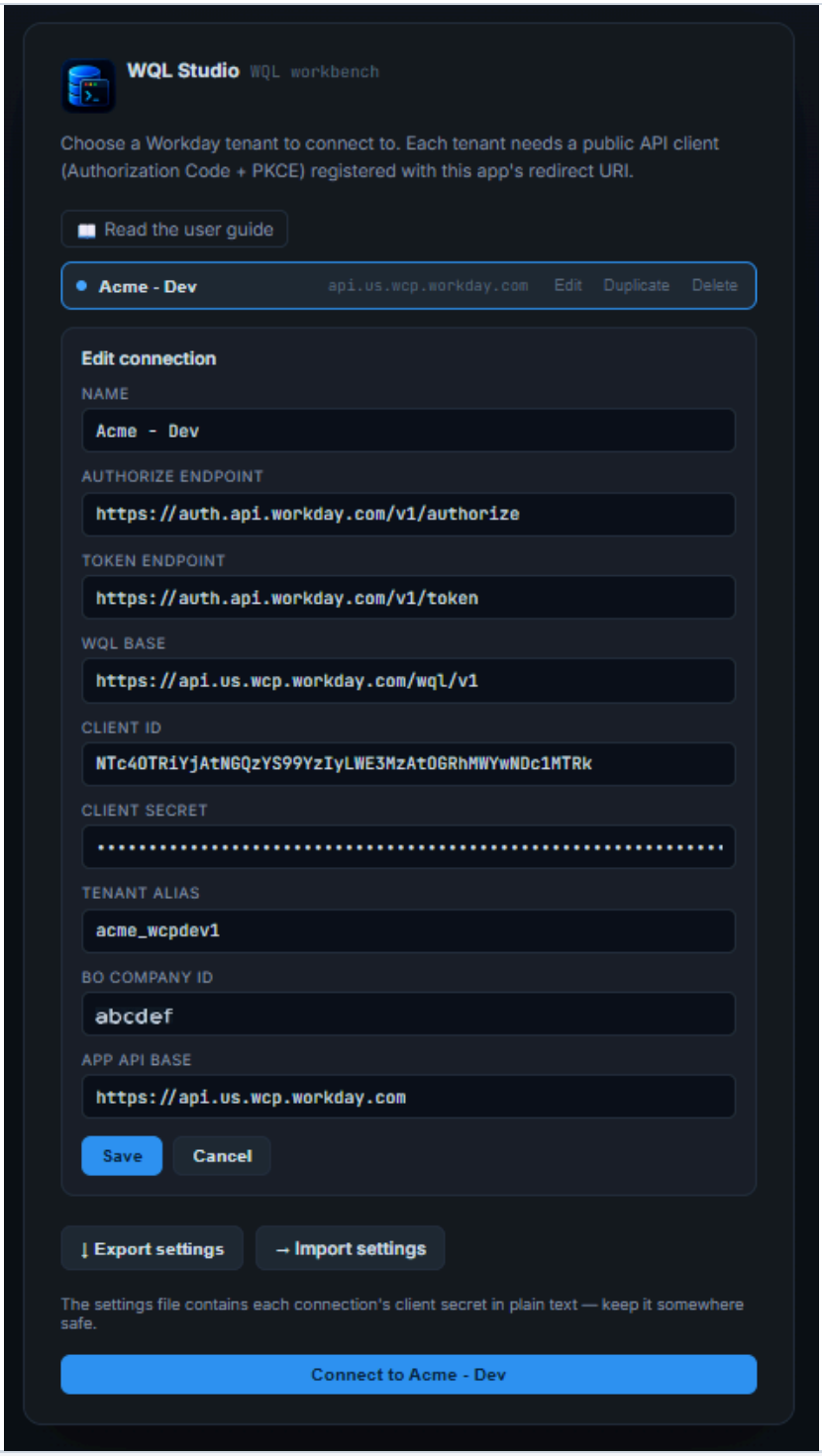
> **Missing a field later?** If a query returns nothing you expect, or a business object isn't listed, the API client's *scopes* are the usual cause, you can edit them on the same page.

2. Add a connection

On the Connect screen, choose **+ Add connection** and fill in the tenant details.



The Connect screen, add, edit, and choose a tenant connection.



The connection form. All nine fields are required.

Field	What it is
`Name`	A label for this tenant (e.g. “Acme WCP Dev1”).
`Authorize endpoint`	The tenant’s OAuth authorize URL.
`Token endpoint`	The tenant’s OAuth token URL.
`WQL base`	The WQL API base, e.g. `https://…/wql/v1`.

Field	What it is
`Client ID`	The API client's ID.
`Client secret`	The API client's secret.
`Tenant alias`	The tenant short name (e.g. `acme_wcpdev1`).
`BO Company ID`	The marker that identifies *your* custom BOs, required to enable Insert/Update/Delete and Export/Import. Example, acmequerytool_*abcdef*_loanApplications
`App API base`	The WCP Extend App API base, e.g. `https://api.us.wcp.workday.com`.

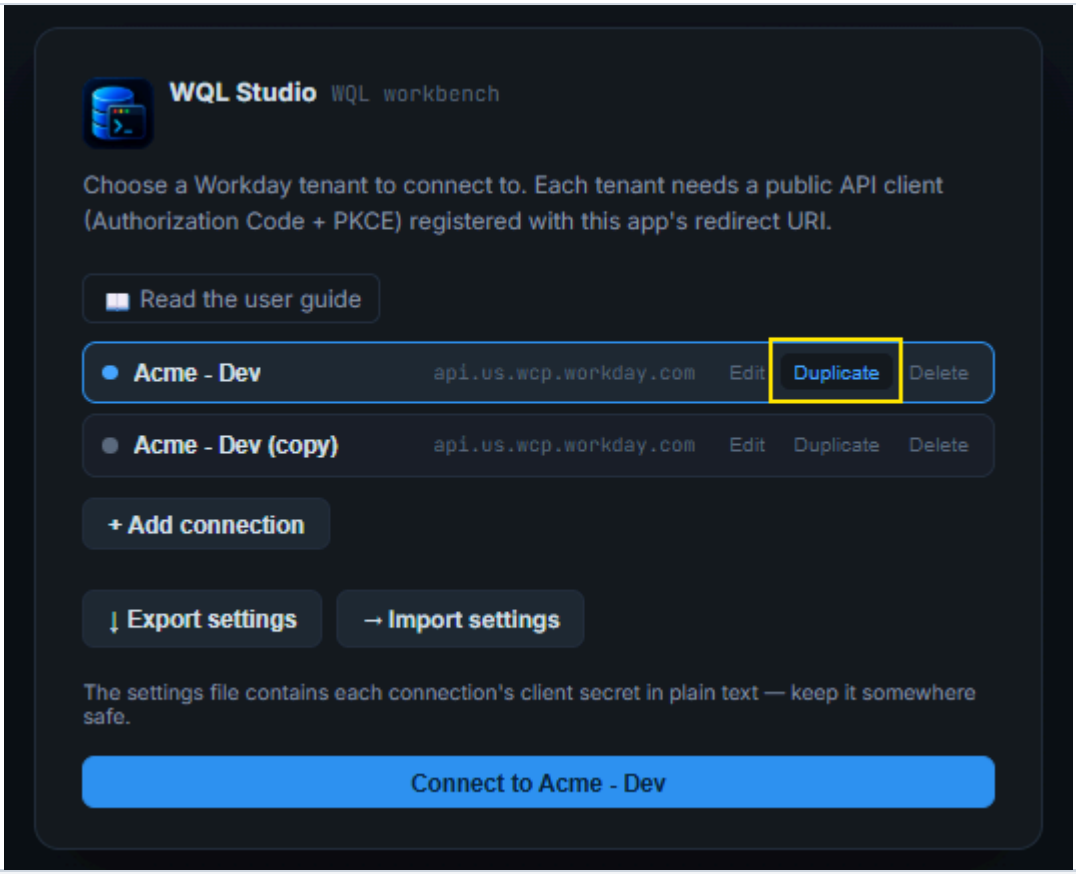
3. Connect

Select the connection so it's active, then click **Connect to <name>**. You'll be sent to Workday to sign in, then returned to the app authenticated.

Managing connections

Each saved connection has three actions:

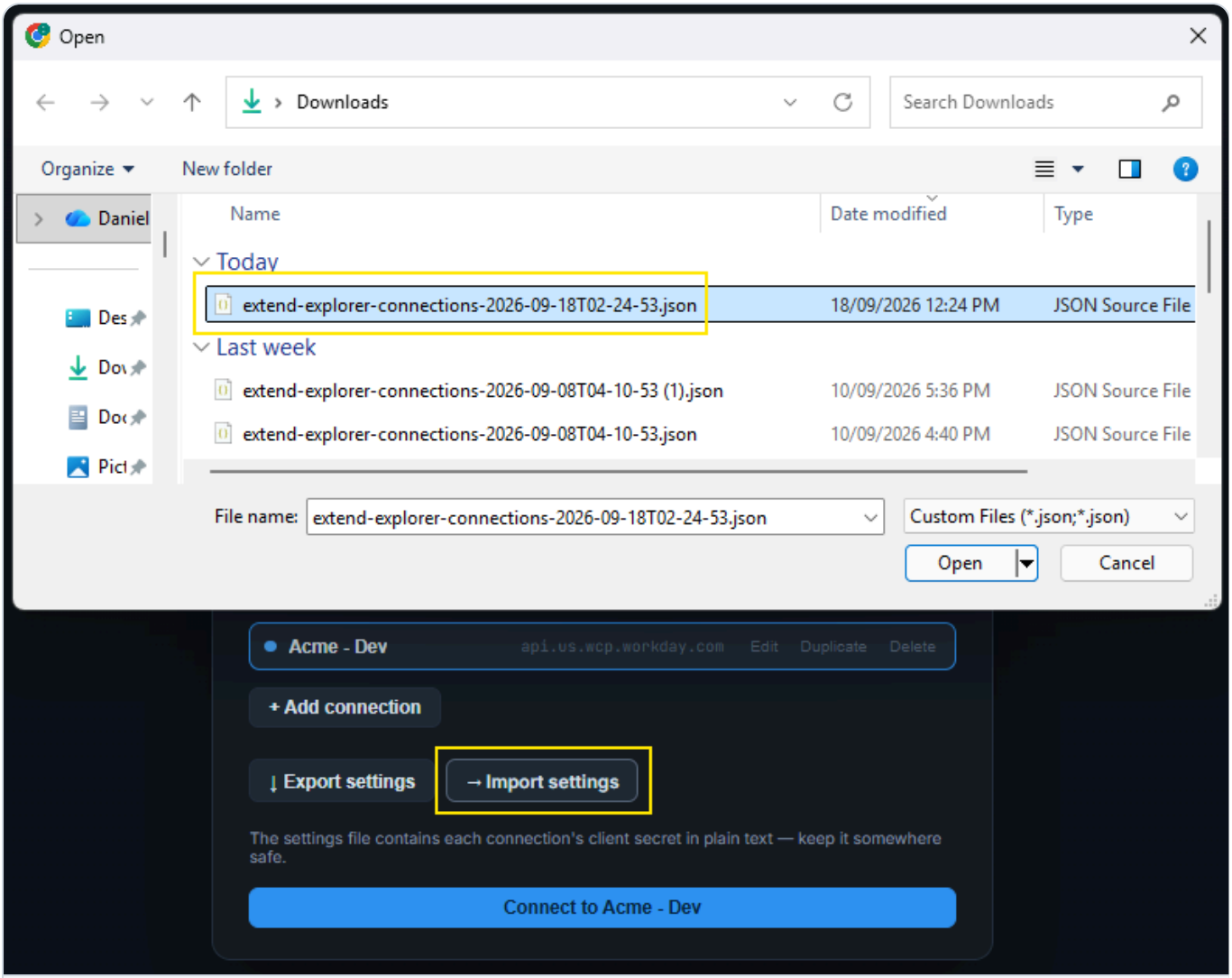
- **Edit**, opens the same form, prefilled. **Save** stays disabled until every field has a value.
- **Duplicate**, copies the connection, a quick start for another tenant on the same data centre.
- **Delete**, asks *"Delete this connection?"* before removing it. This only removes the saved settings from your browser; nothing changes in Workday.



Saved connections with Edit, Duplicate and Delete, and the delete confirmation prompt.

Import / export settings

Use **Export settings** to download all your connections as a JSON file, and **Import settings** to load them on another machine or share with a colleague. Importing adds the file's connections to your list and reports how many were read and saved (e.g. *"Imported 3 connections, 3 saved."*).

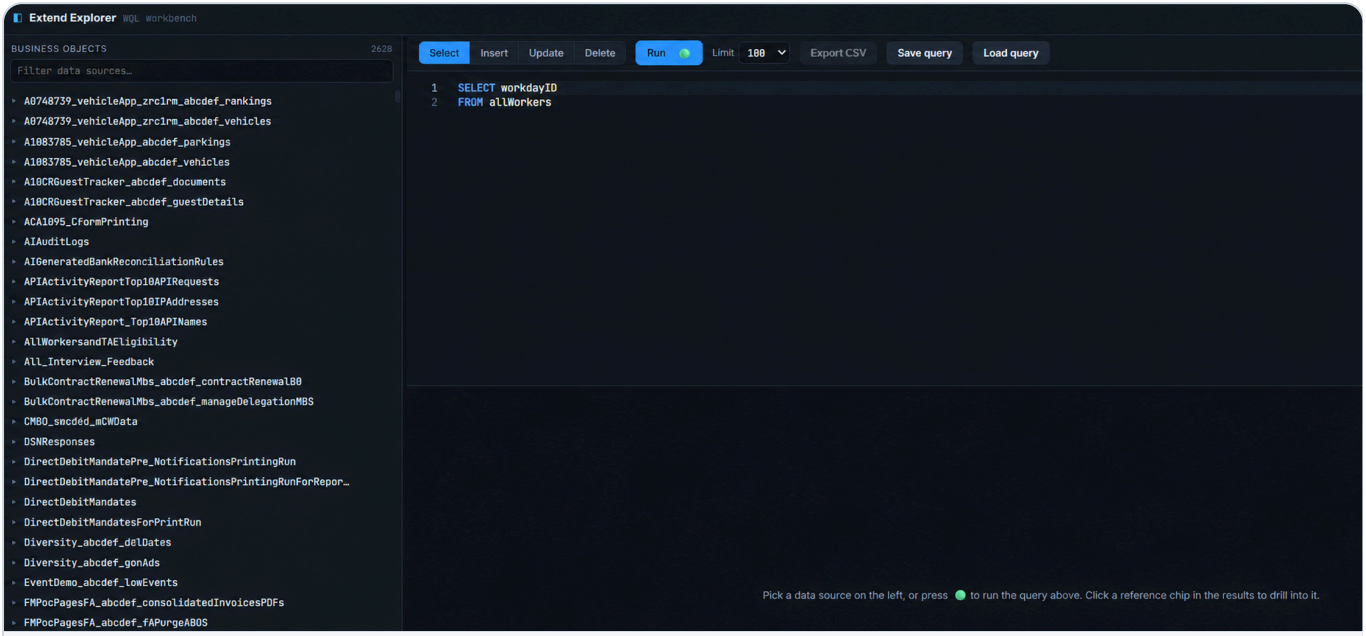


Export / import all saved connections as a settings file.

> **The settings file contains client secrets in plain text.** Keep it somewhere safe and import it when required.

The Explorer

Once connected, the workbench has three regions: the **schema tree** on the left, the **query editor** in the middle, and the **results** below it. The toolbar across the top switches modes and runs queries.



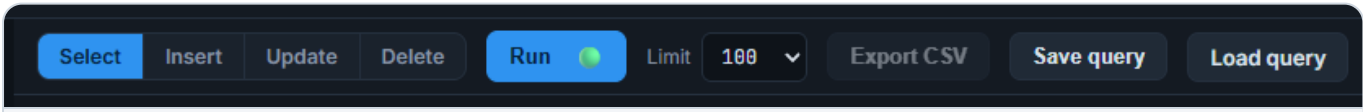
The Explorer layout: schema tree, toolbar, editor, and results.

Resizing the panes

- Drag the **vertical divider** between the tree and the main area to widen or narrow the schema tree.
- Drag the **horizontal divider** between the editor and the results to trade space between them.

Both sizes are remembered across reloads.

The toolbar



The toolbar: mode switch, Run, Limit, Export CSV, Save/Load query, and status.

- **Mode switch**, `Select` `Insert` `Update` `Delete` `Export/Import`.
- **Run** 🟢, runs the query.
- **Limit**, rows per page (10 / 25 / 50 / 100). 100 is the API's maximum page size; to get more rows out in one go, use **Export CSV** or Export/Import mode, which page through everything for you.
- **Export CSV**, exports the full result set (not just the current page).
- **Save query / Load query**, save the editor's WQL to a `.wql` file, or load one back.
- **Status**, rows returned and how long the query took.

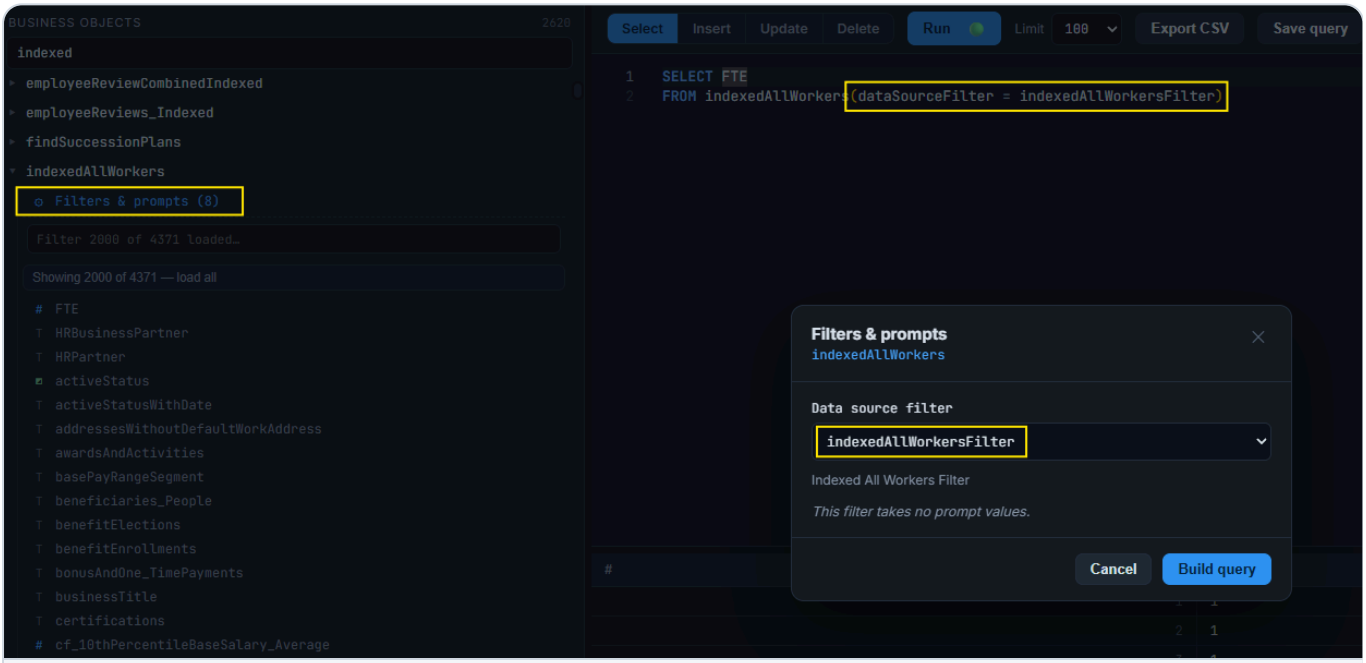
Select, querying data

Select mode runs read-only WQL against any business object. It's where you'll spend most of your time exploring the data model.

Browsing the schema tree

The tree lists queryable data sources. Filter it with the search box, then:

- Click **use** on a source to start a query from it.
- Expand a source to see its **fields**, each with a type glyph (`→` reference, `⋮` multi-instance, `#` numeric, `◷` date, `◩` boolean, `T` text).
- Click a **field** to insert its alias at the cursor.
- Sources with a `⚙ Filters & prompts` entry let you build a prompted FROM clause.



The schema tree, data sources, fields with type glyphs, and filters.

Filters & prompts

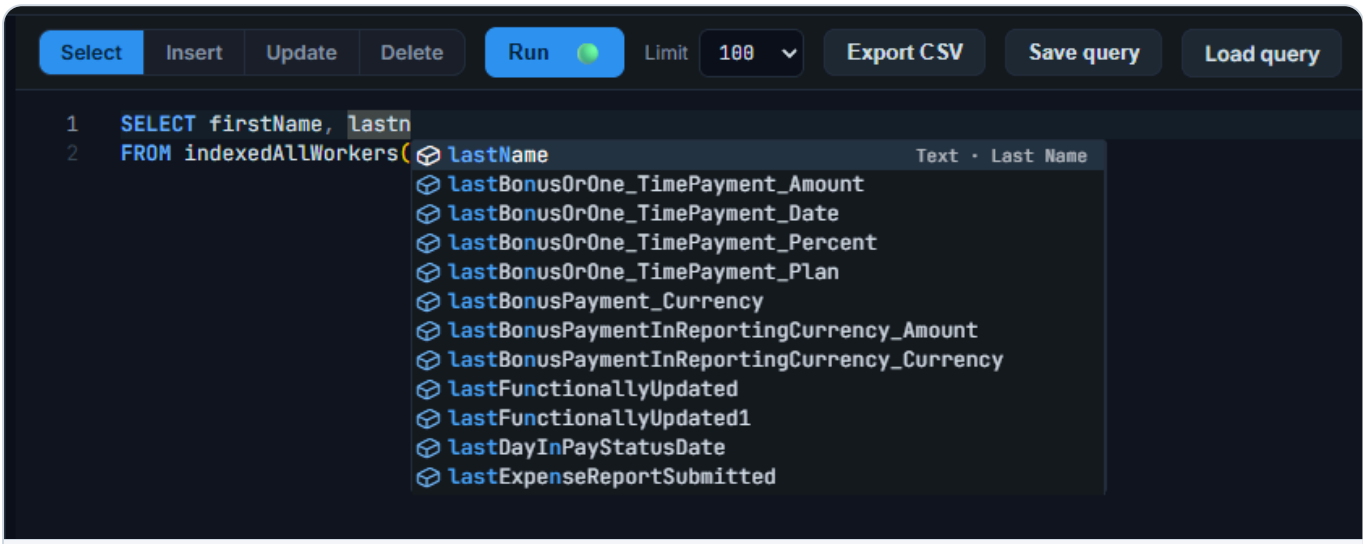
Clicking `⚙ Filters & prompts` opens a dialog for building a prompted `FROM` clause:

1. If the source has more than one **data source filter**, choose one from the dropdown. Its description appears underneath.
2. Fill in the **prompts**. Each shows its type; required ones are marked `*`. Booleans get a true/false dropdown, dates a date picker, and instance prompts expect a WID.
3. Click **Build query** (disabled until every required prompt is filled). The editor is filled with a query using that `FROM` clause, ready for you to add fields.

> **Big BOs.** Very wide objects (like `allWorkers`) load their first couple thousand fields immediately; use **"Showing N of M, load all"** to pull the rest.

Writing WQL

The editor has WQL syntax highlighting and schema-aware autocomplete, press `Ctrl` + `Space` to see data sources, fields, and keywords. Run with **Run** 🟢.



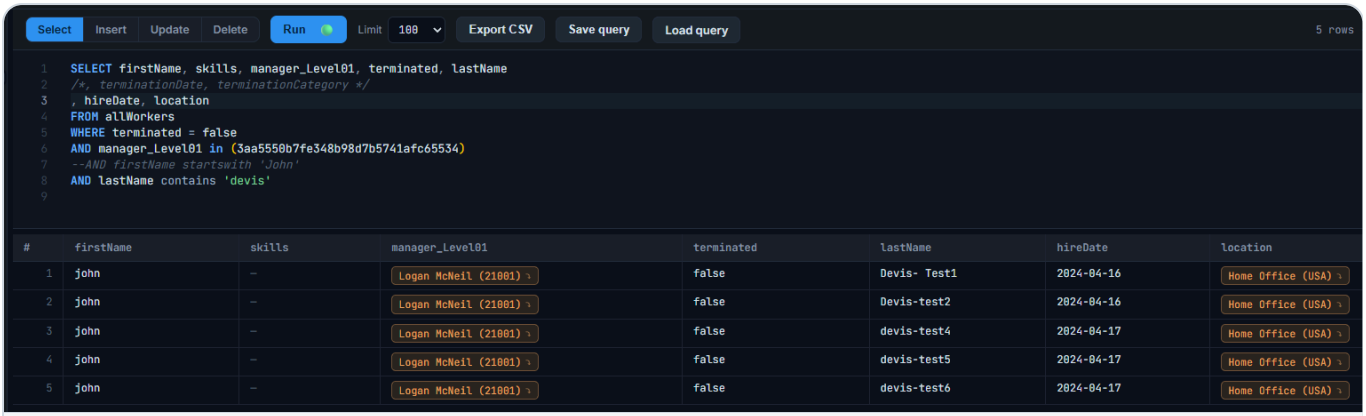
Autocomplete suggesting fields for the source in the FROM clause.

```
SELECT workdayID, legalName, primaryWorkEmail
FROM allWorkers
```

Comments

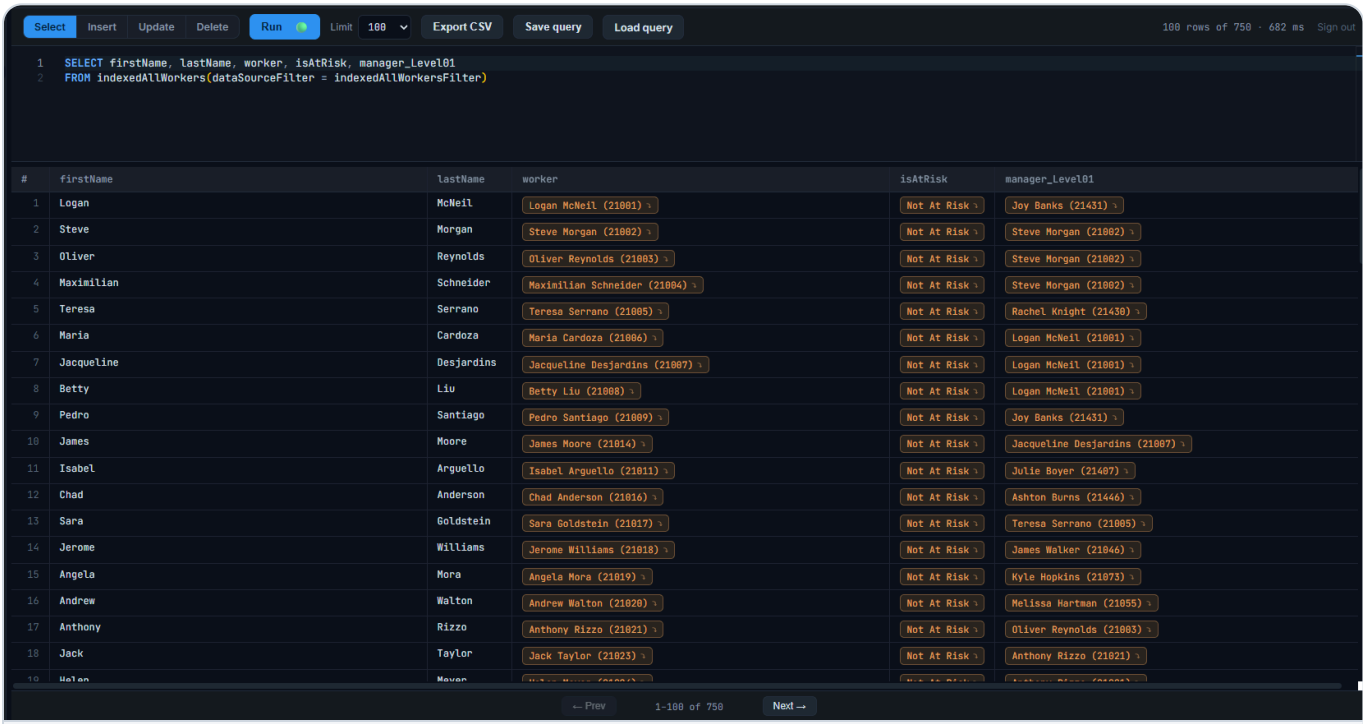
The editor supports SQL-style comments, which are stripped out before the query is sent to Workday, handy for toggling fields or clauses on and off while you iterate:

- `--` comments out the rest of a line
- `/* … */` comments out a block



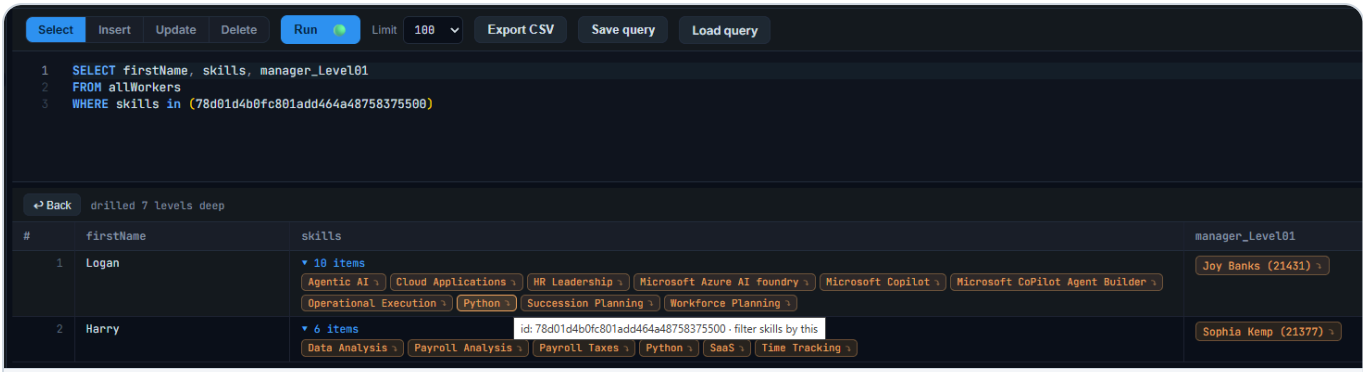
Line (--) and block (/* */) comments in the editor; both are ignored when the query runs.

Reading results



The results grid with reference chips and multi-instance cells.

- **Reference chips**, a linked instance. Click a chip to **drill in**: the app re-queries the same source filtered to that instance, keeping your select list. Use **↩ Back** to return. `Ctrl`-click instead to *add* that reference as a filter on your current query, keeping everything you've written (clicking several chips in one column merges them into a single `in (…)` list).
- **Multi-instance cells**, click to expand the list of items.
- **Paging**, **Prev / Next** move through pages at your chosen Limit.



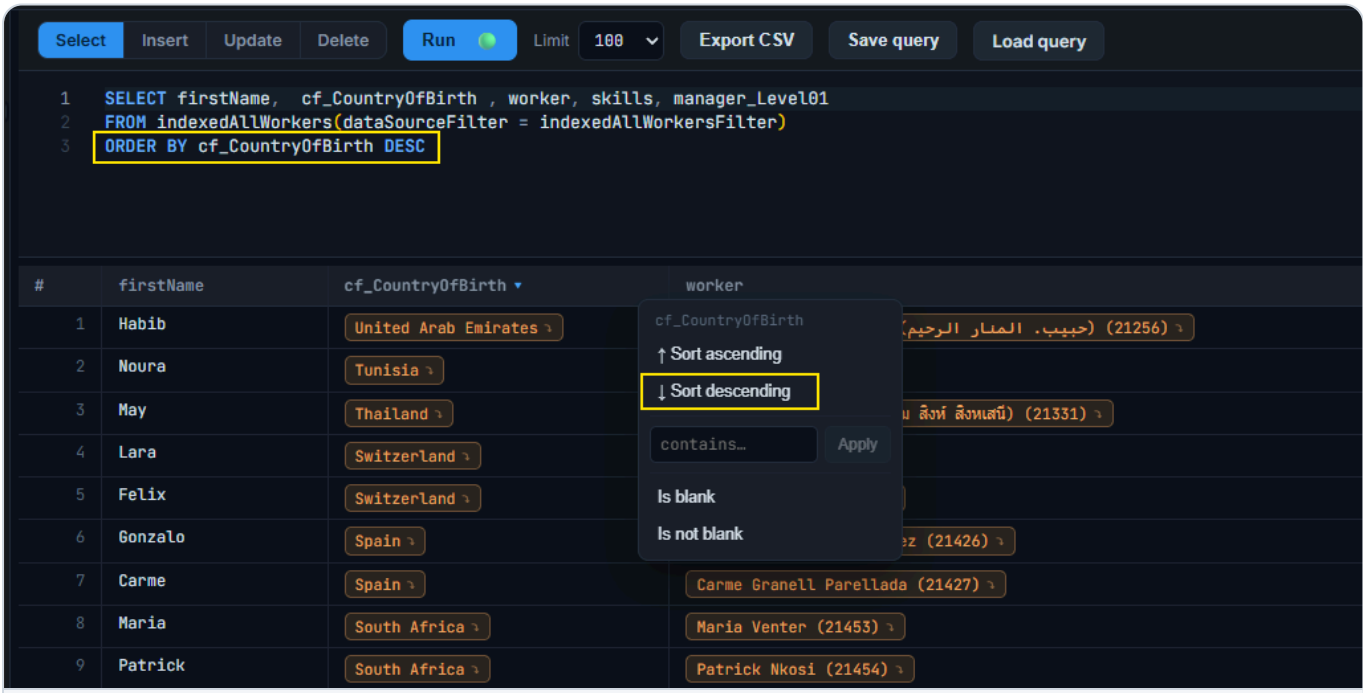
Drilling into a referenced instance from a chip.

Sorting & filtering columns

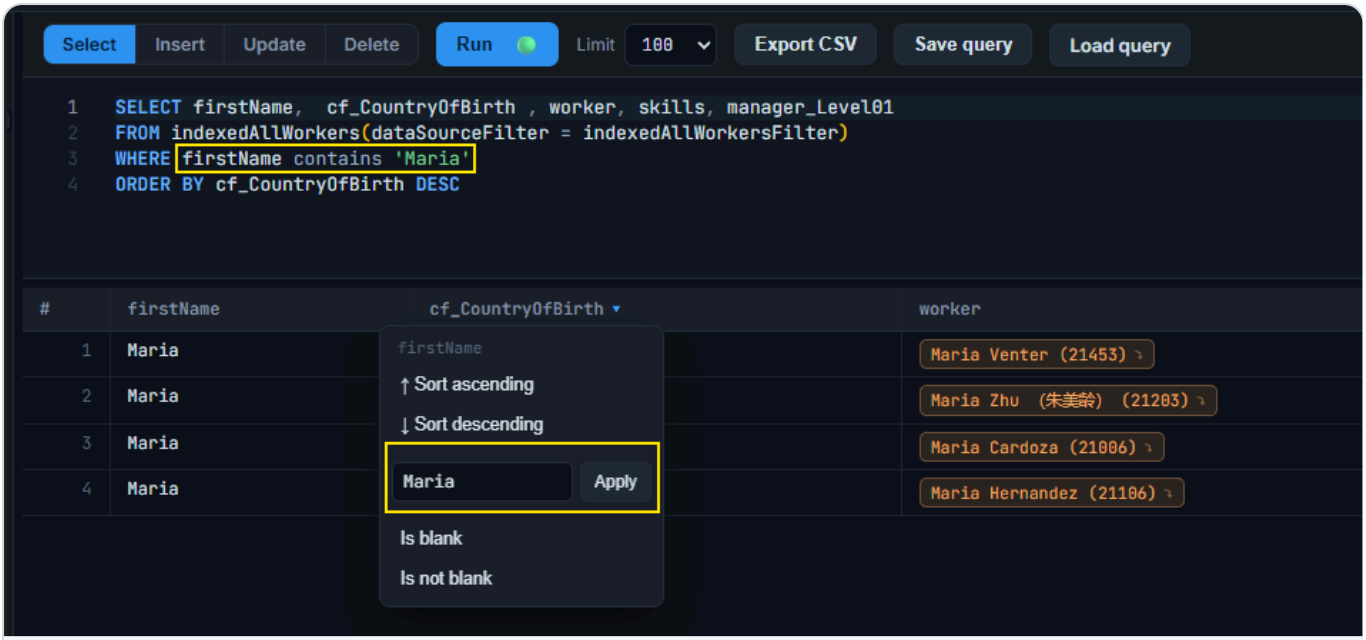
Hover a column header and click the `···` menu to sort or filter without editing the WQL by hand. The options match the field's type:

- **Sort ascending / descending**, on any column.
- **Text**, *Contains…*, plus *Is blank* / *Is not blank*.
- **Boolean**, a `true` / `false` dropdown (WQL compares booleans with `=`).
- **Numeric & date**, a comparison (`=`, `>`, `>=`, `<`, `<=`) with a value, plus blank / not blank.
- **Instance fields**, blank / not blank; filter these by `Ctrl`-clicking a chip in the grid instead.

Each action rewrites the query and re-runs it. Re-applying on the same column replaces its previous filter rather than stacking a second one.



The column header menu, sort, contains, and blank filters.



Applying a “contains” filter rewrites the WHERE clause and re-runs the query.

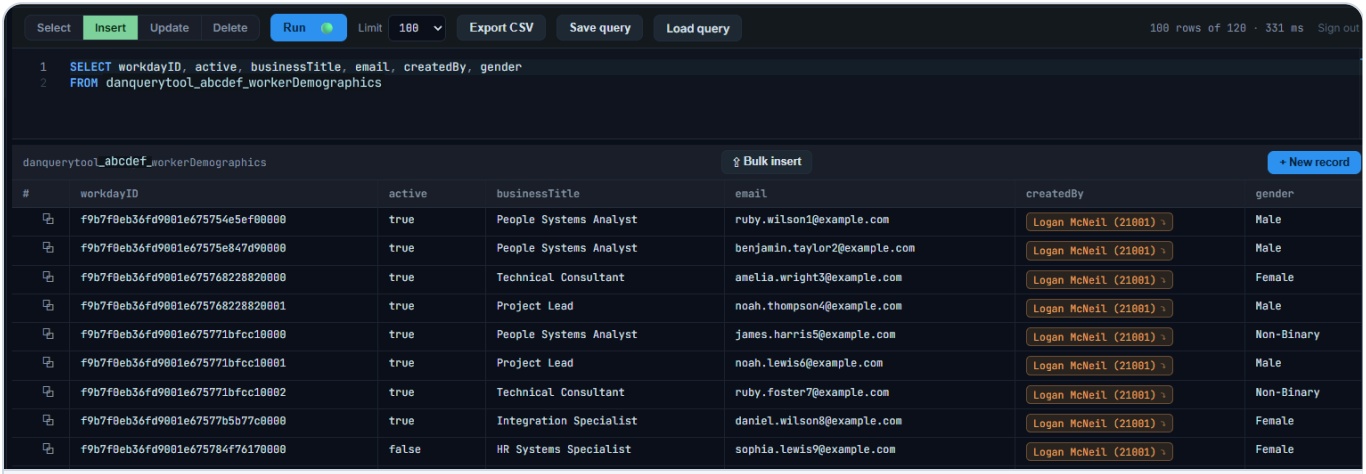
Exporting & saving

Export CSV streams the entire result set to a file. **Save query** / **Load query** keep a library of WQL you can reuse or share.

Insert

> **Custom BOs only.** Insert / Update / Delete are enabled only for business objects that carry your `BO Company ID`. Set that on the connection, then switch to a write mode and the tree filters to just your custom BOs.

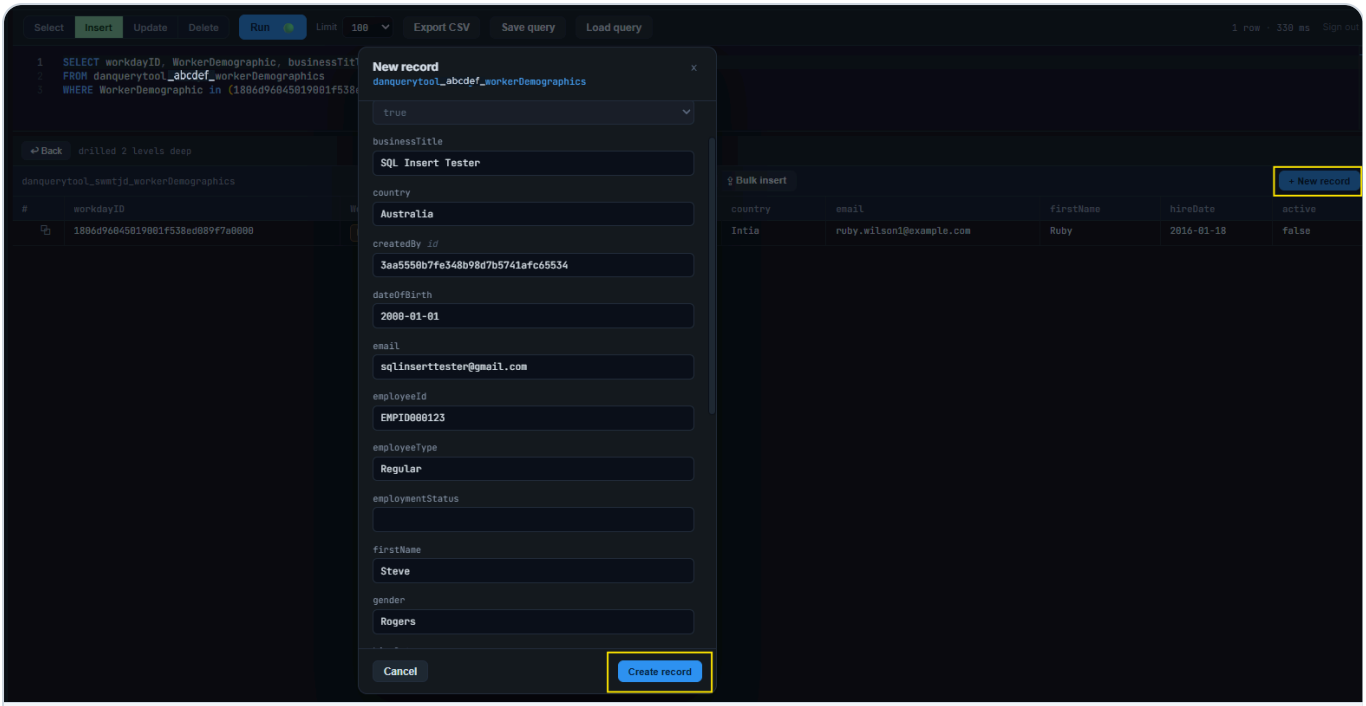
Switch to **Insert** mode, pick a custom BO, and run to see its current rows. Then create records one of three ways.



Insert mode on a custom BO, clone buttons on each row, plus New record and Bulk insert.

New record

Click **+ New record** to open a form built from the BO's writable fields. Fill what you need, empty fields are omitted so Workday applies its defaults and required-field rules. Read-only / derived fields that Workday rejects are dropped automatically and the insert retried.



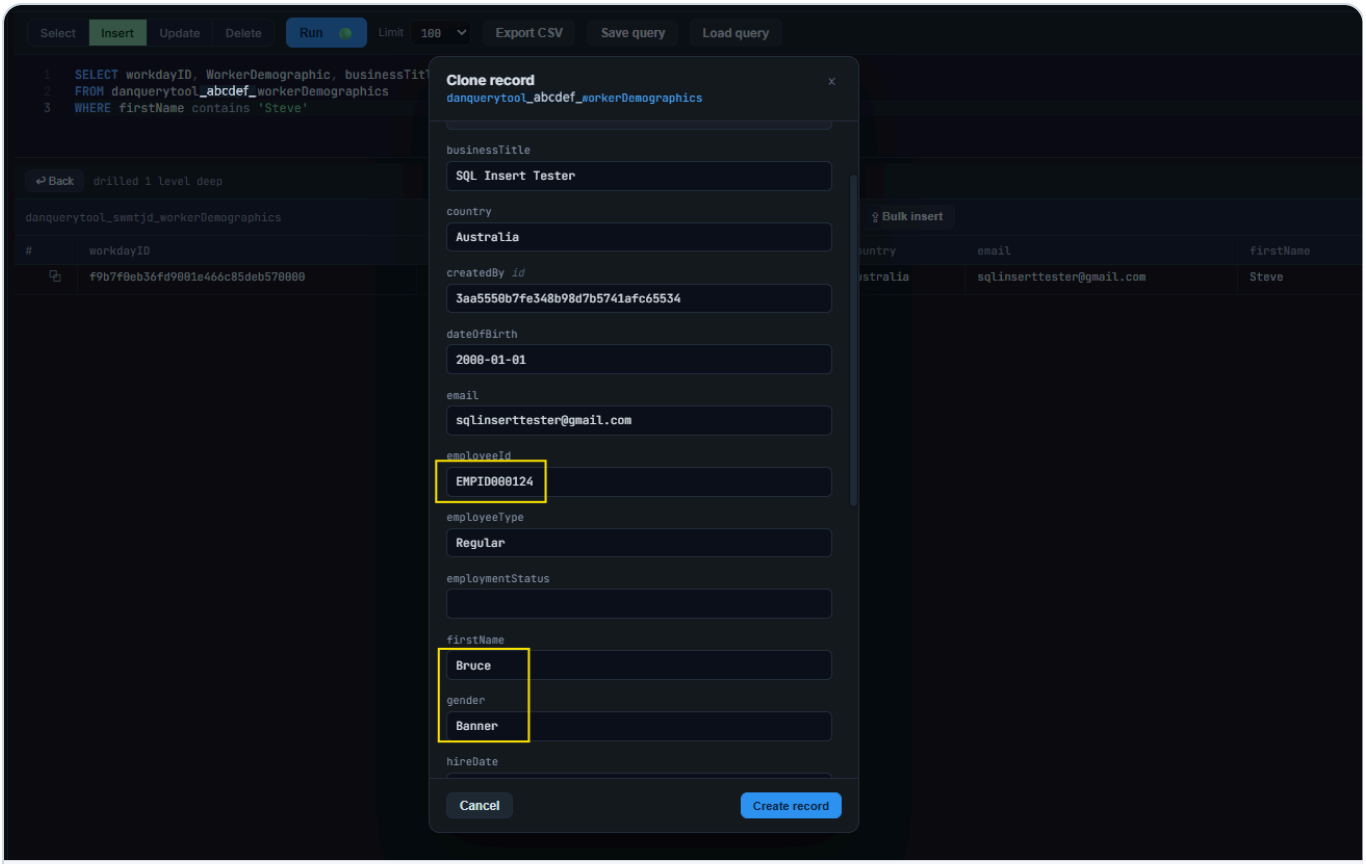
The New record form, with fields typed from the BO schema.



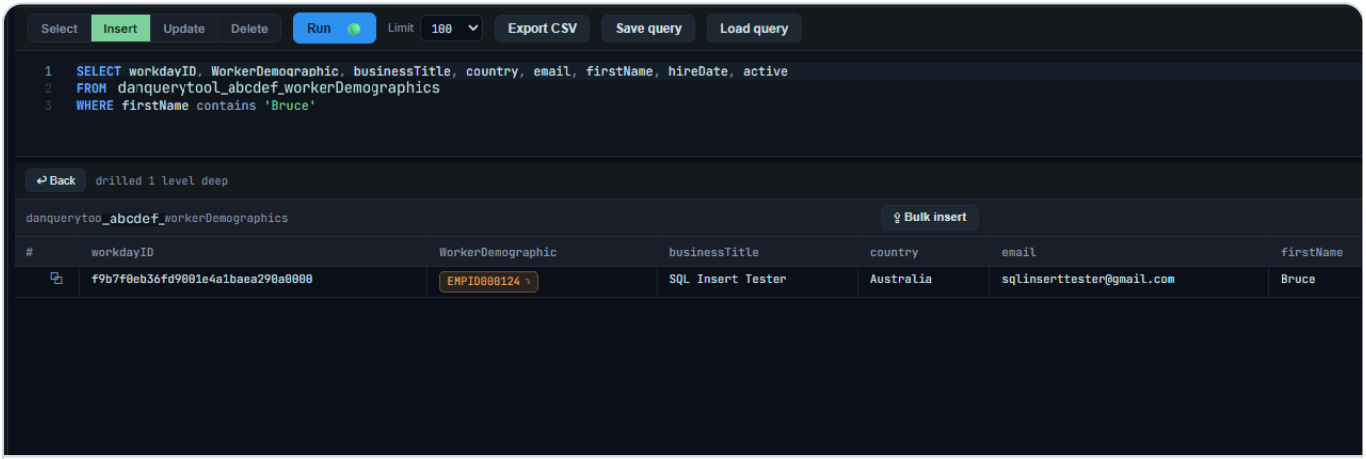
The newly created record, returned by re-running the query.

Clone a row

In the results grid, the clone button ( ) on a row prefills a new record from that row's values, handy for creating variants.



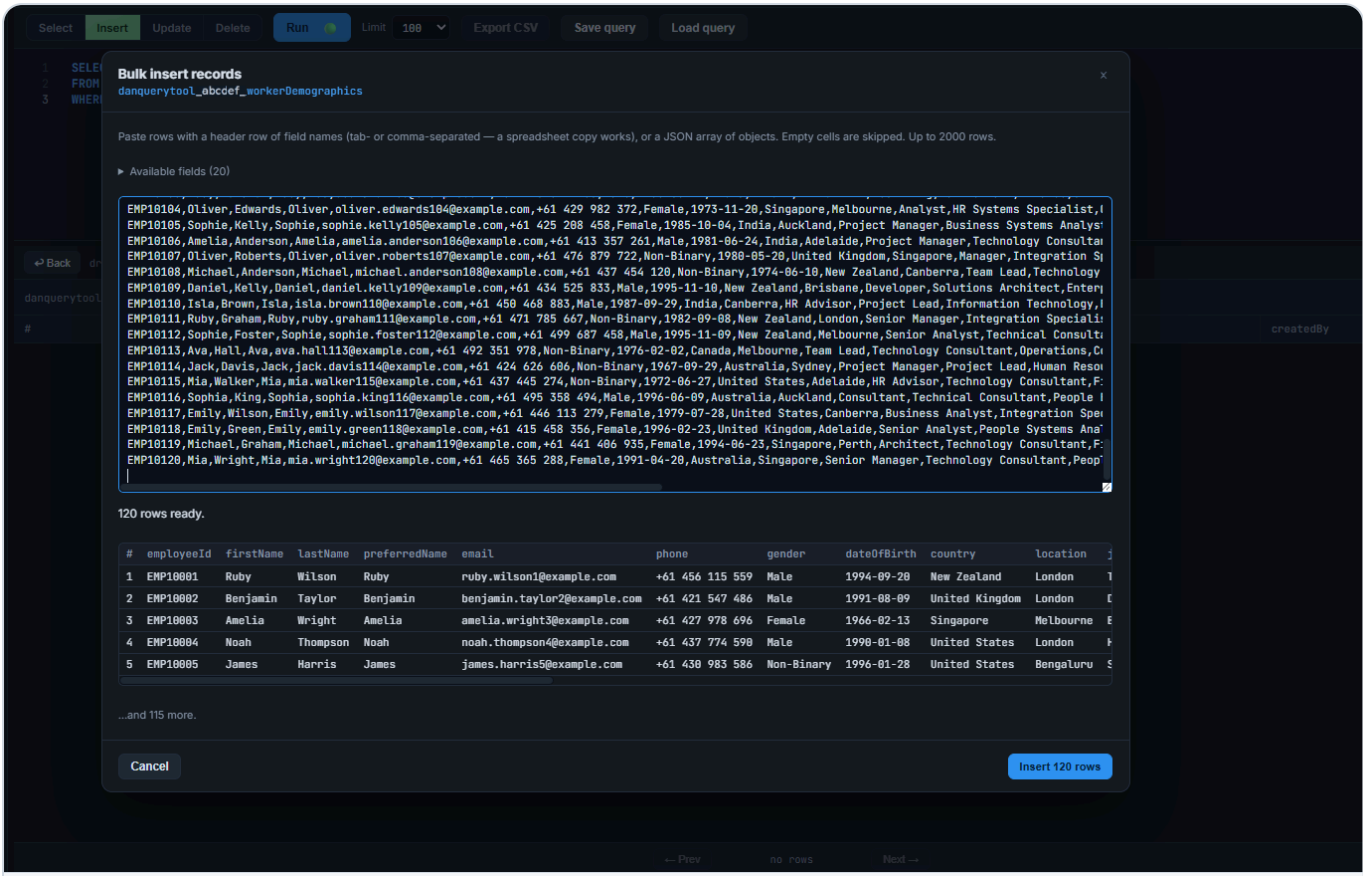
Cloning a row prefills a New record form from that row's values.



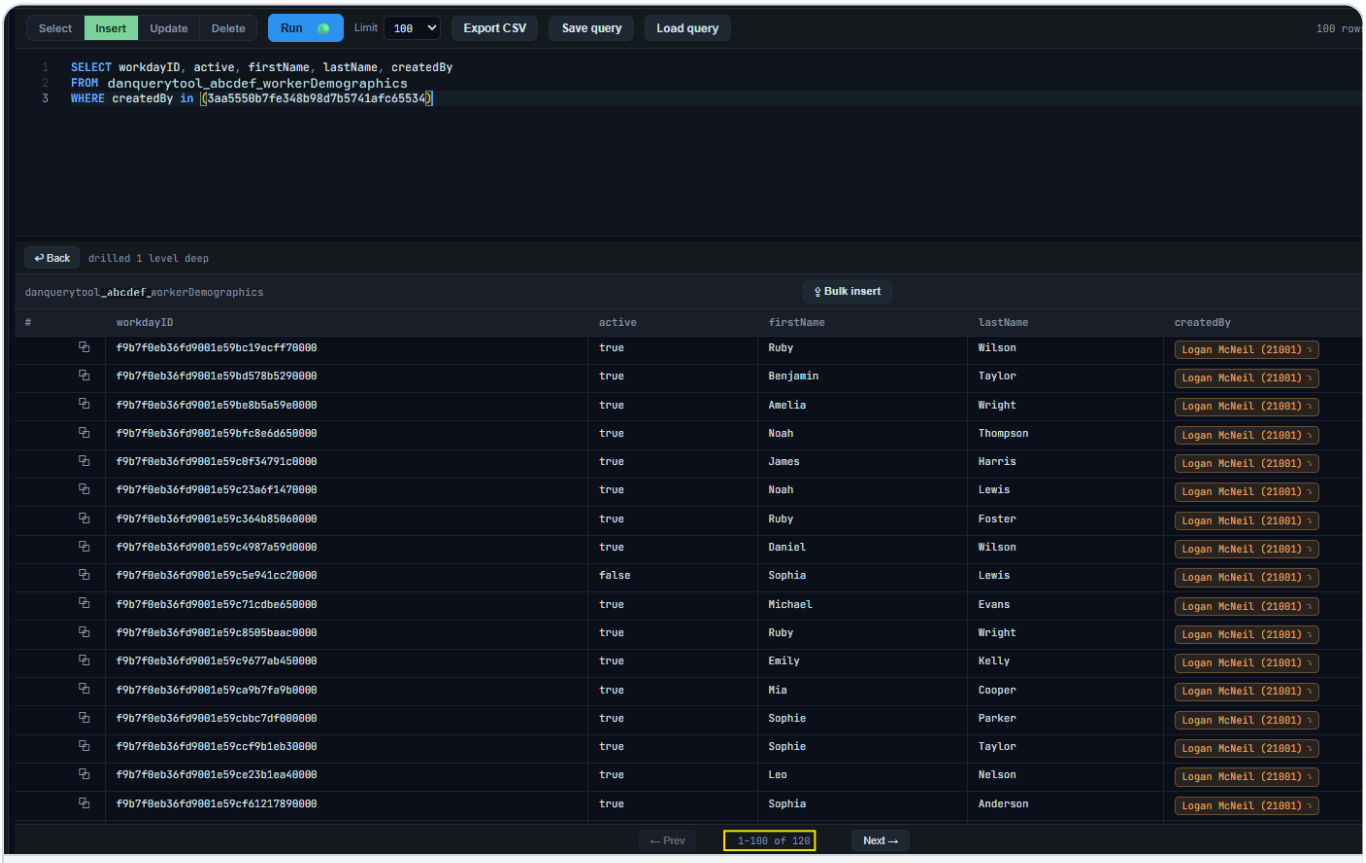
The cloned row, created as a separate new instance.

Bulk insert

⇪ **Bulk insert** lets you paste many rows at once, either tab/comma-delimited with a header row (a spreadsheet copy works), or a JSON array of objects. You'll see a preview and a per-row result; rows are sent in chunks of 100.



Bulk insert, paste rows (here 120), preview, then insert in batches of 100.



Bulk insert, the pasted rows are inserted

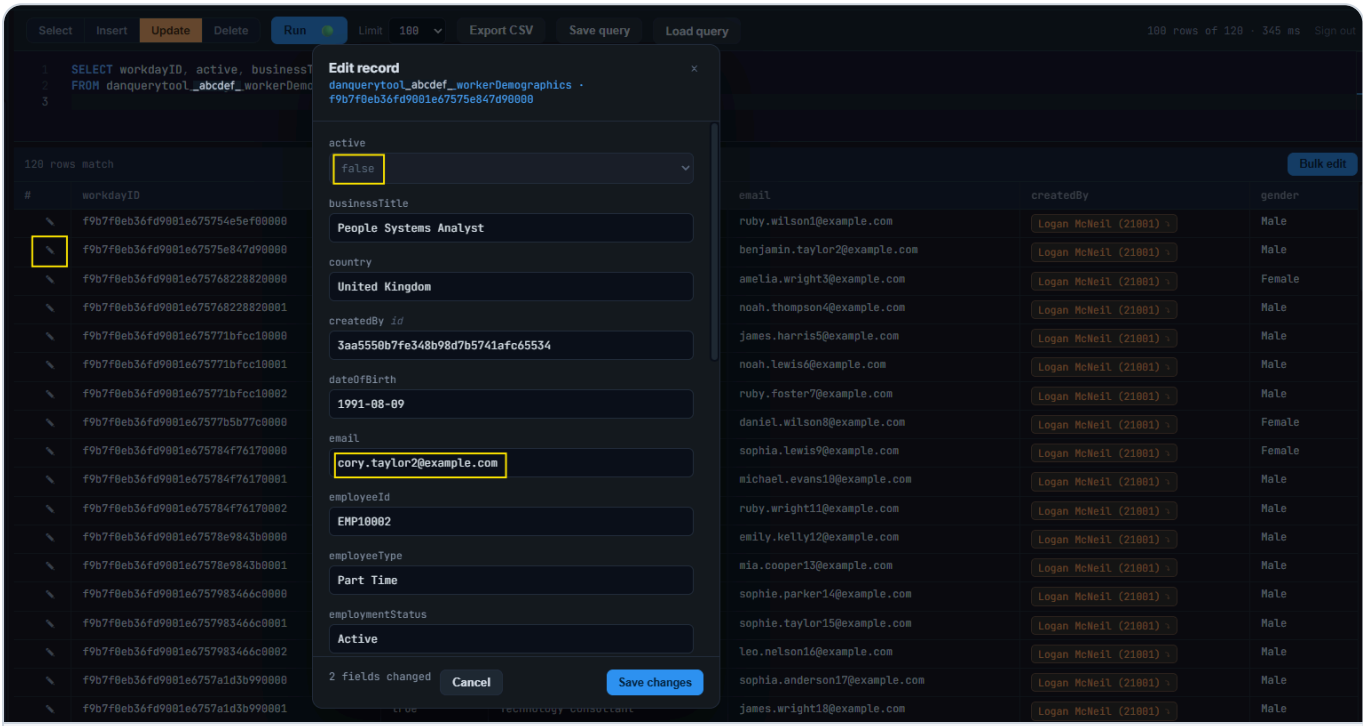
If Workday rejects some rows, the rest are still created. Each rejected row is listed with Workday's message, so you can fix just those rows and paste them again.

Update

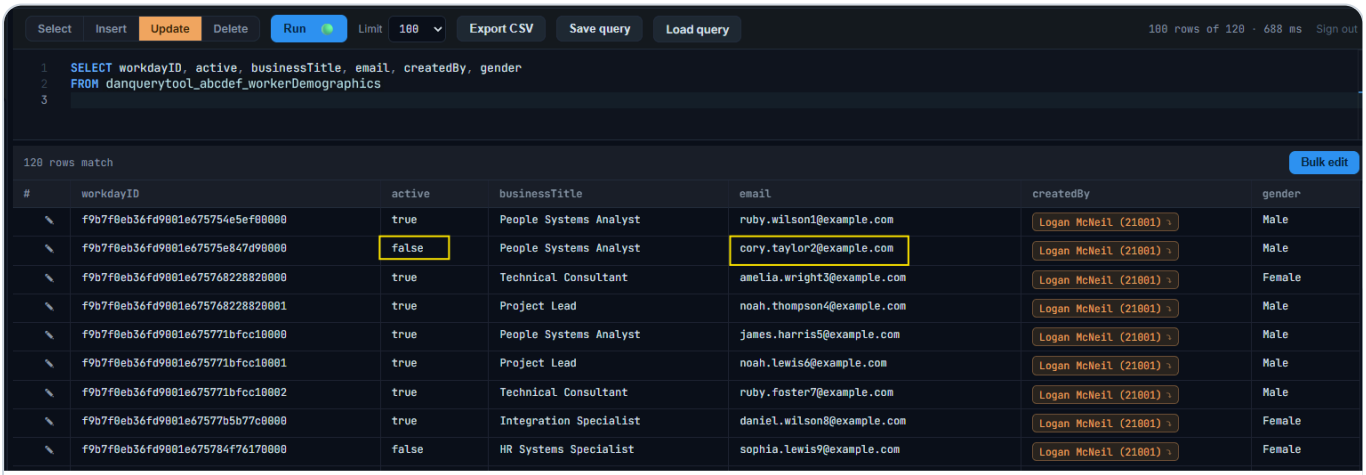
Switch to **Update** mode on a custom BO and run a query to list the rows you want to change. Each row gets an edit pencil, and a **Bulk edit** action appears for the whole result set.

Edit one record

Click the pencil ( ) on a row to open its editor. Change the fields you need, only the fields you actually change are sent back to Workday.



The single-record editor, only the fields you change are sent (here, 2 changed).

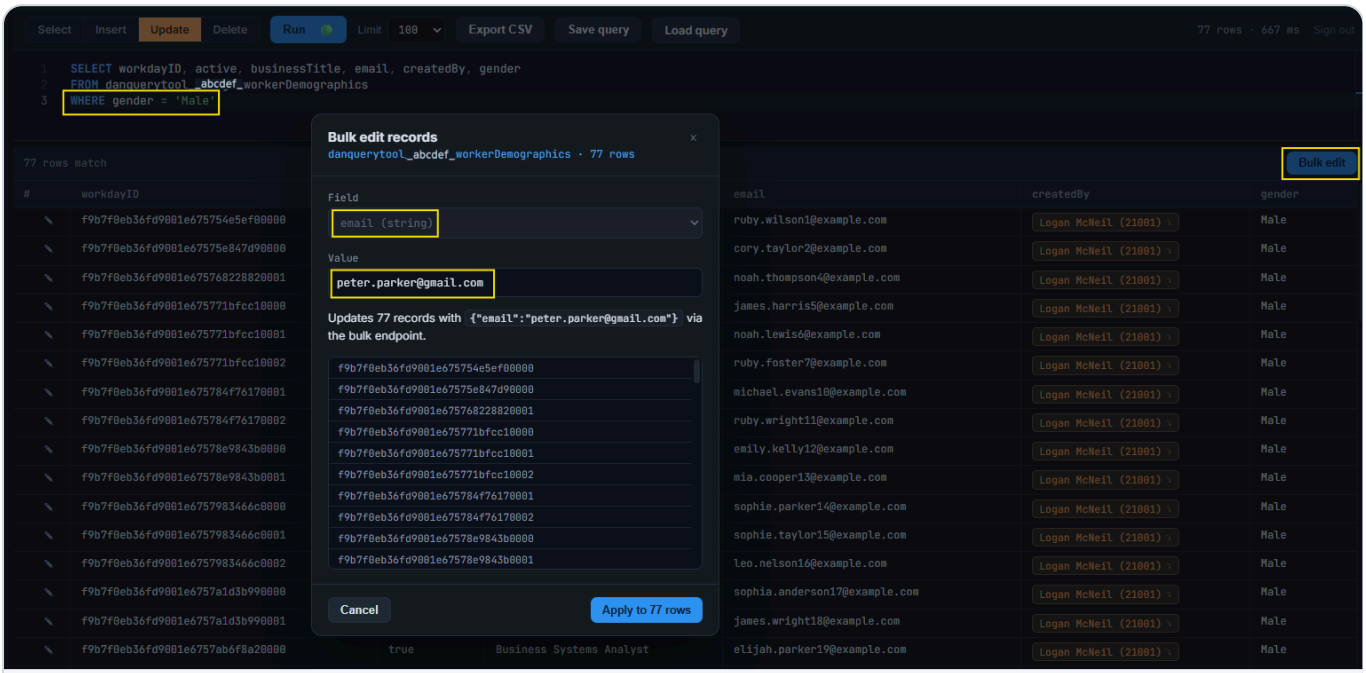


Back in the grid, the row reflects the saved changes.

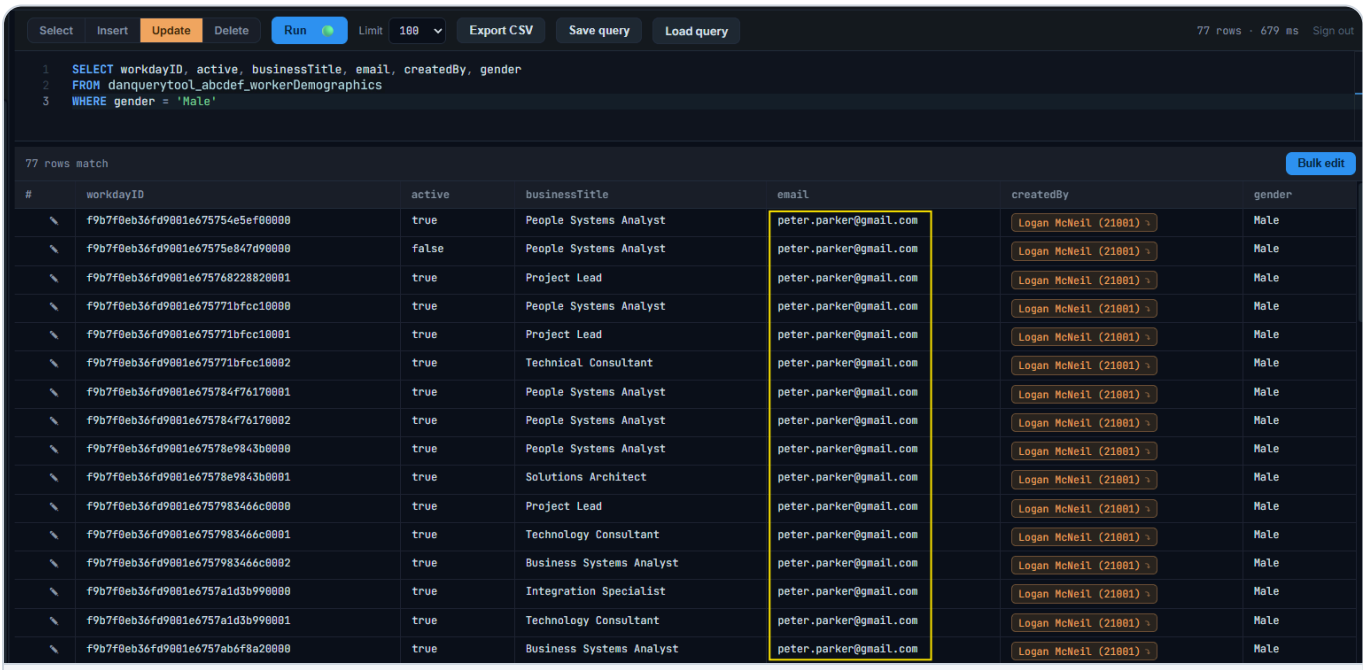
Bulk edit

Bulk edit sets one field to one value across every row your query matched, similar to an `UPDATE table SET field = value WHERE criteria`. The button shows *Collecting…* while it gathers the matching ids across all pages, then the dialog opens and the change is applied in batches of 100.

A single bulk edit touches at most **1,000 rows**. If more match, the dialog doesn't open; a message appears next to the button instead, narrow your `WHERE` and run again.



Bulk edit, pick a field and value; a preview shows the exact JSON and the ids affected.



After applying, every matched row shows the new value.

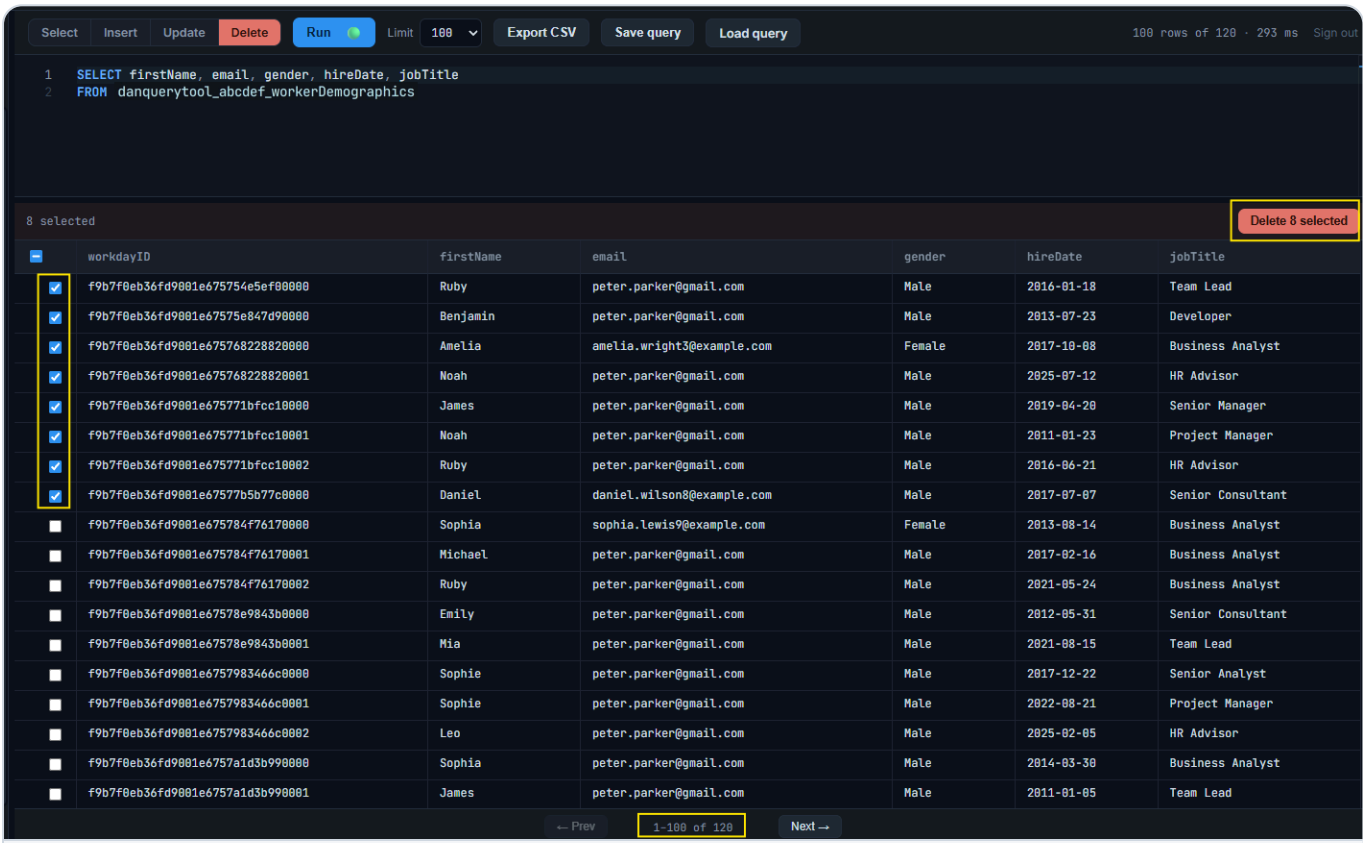
Delete

> **Deletes are permanent.** There's no undo, the records are removed via the API.

Switch to **Delete** mode on a custom BO and run a query. The bar above the grid offers two ways to delete.

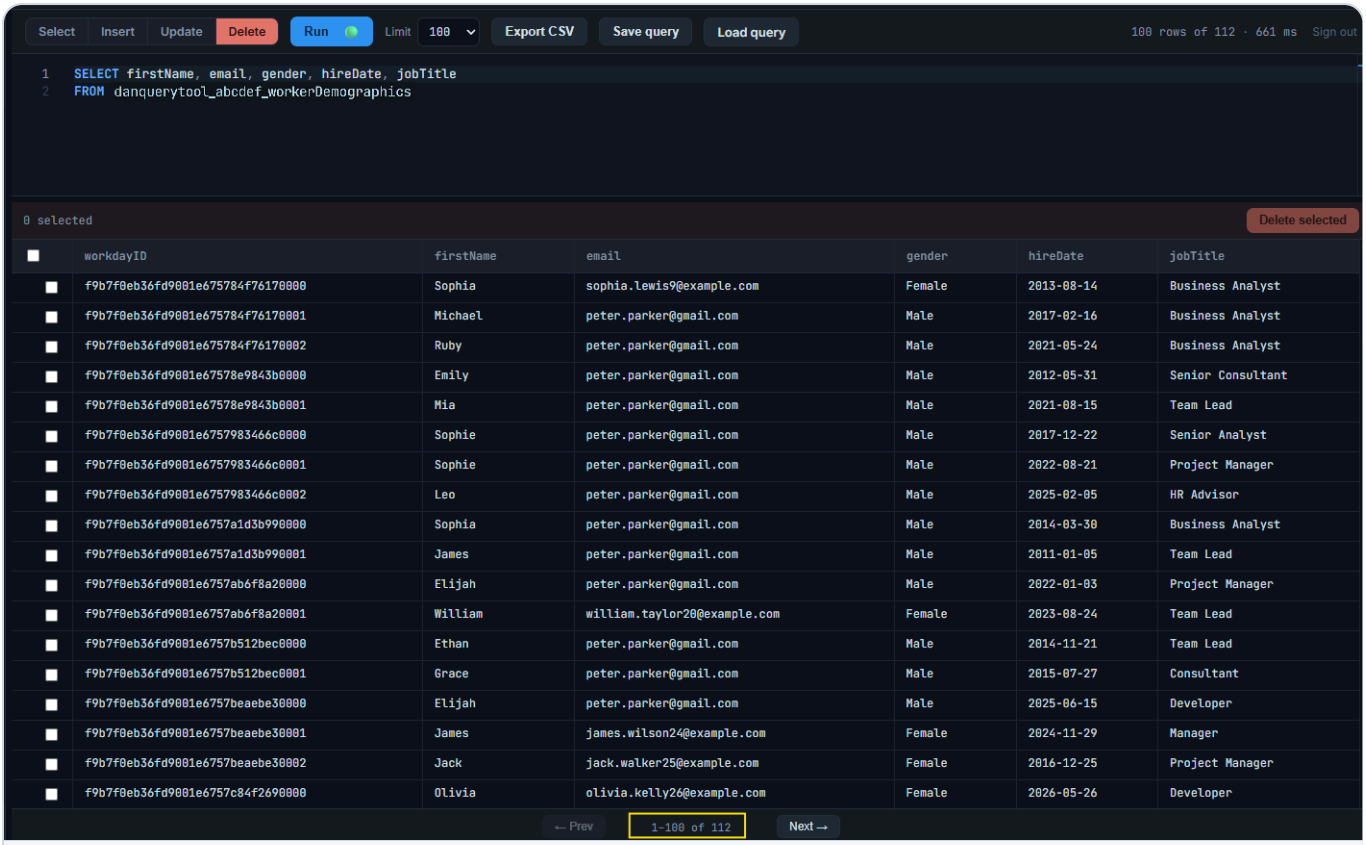
Delete selected

Each row gets a checkbox; the header checkbox selects the whole page. Your selection is kept as you page with Prev / Next, so you can pick rows from several pages. Then click **Delete N selected**.



Delete mode, select rows with the checkboxes.

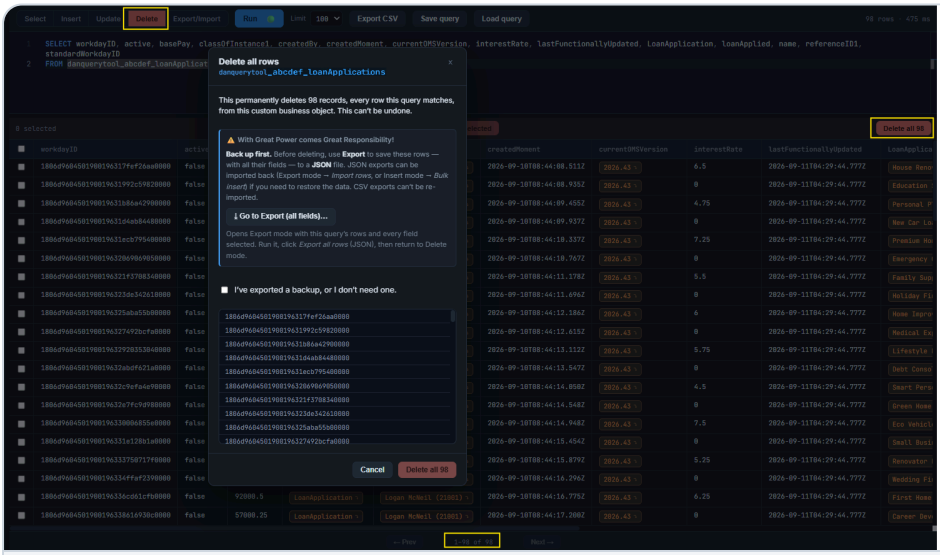
A confirmation shows how many records will go. Nothing is sent until you click **Delete**. Afterwards you get a per-row result showing what was deleted and what failed, and the grid refreshes.



The delete confirmation and per-row results.

Delete all

Delete all N deletes *every row your query matches*, across all pages, not just the one on screen. N is the query's total match count.



Delete all shows the full match count (here 112), not the page size.

When you click it, the app collects the ids of every matching row (*Collecting…*), then opens the **Delete all rows** confirmation. It has extra safeguards:

- A **backup warning** recommending you export the rows to **JSON** first, so they can be imported back.
- ↓ **Go to Export (all fields)…**, closes the dialog, switches to Export/Import mode, and loads your query with *every field* of the BO selected (same `FROM` / `WHERE`, so the same rows). See the walkthrough.
- A tick-box, *"I've exported a backup, or I don't need one."*, which must be ticked before the Delete button enables.

Screenshot

The Delete all confirmation, backup warning, Go to Export, and the acknowledgement tick-box.

`guide/delete-all-confirm.png`

Delete all is capped at **1,000 rows**. If more match, a message appears next to the button instead of the dialog, narrow your `WHERE`.

Reading the results

Both kinds of delete are sent in batches of 100 and end with the same per-row list. A row that fails (for example, one already deleted by someone else) shows Workday's message; the rest of the batch is unaffected.

> **Back up first.** Before a delete you can't undo, save the rows to JSON, Delete all's *Go to Export* button does the setup for you, or follow the backup & restore walkthrough.

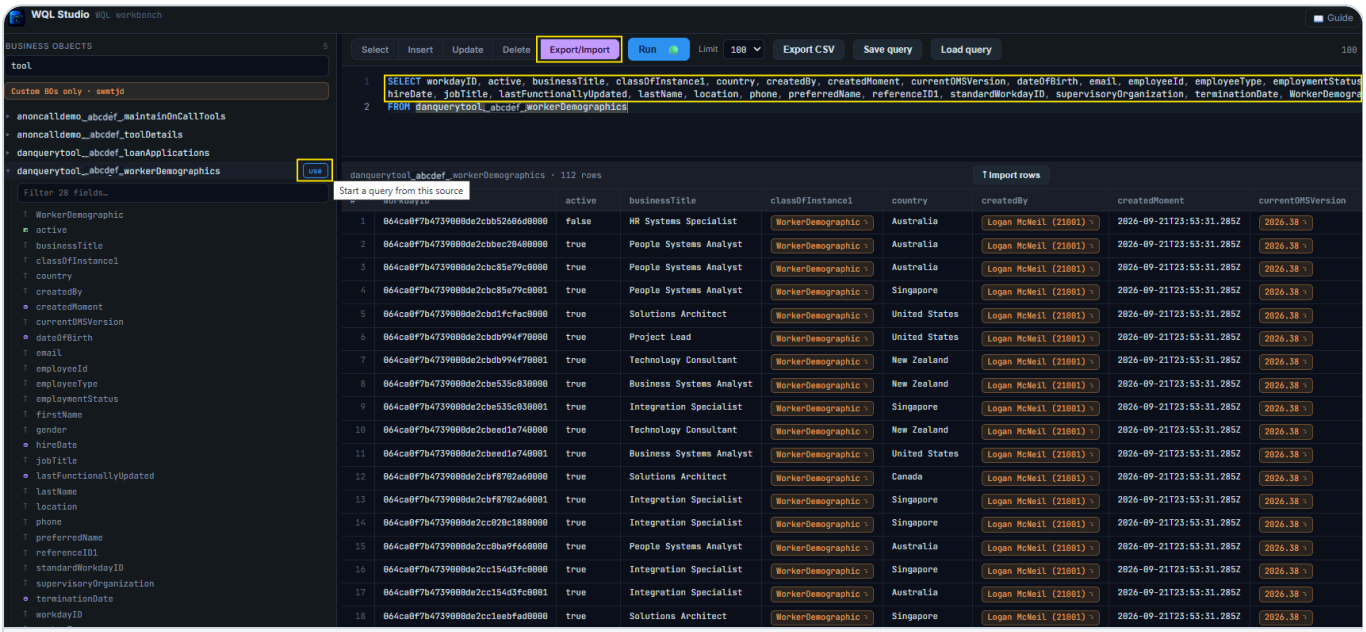
Export / Import

Save every row of a custom BO to a file, and load rows back from one, a backup before a destructive change, a way to move sample data between tenants, or just a snapshot of a data set.

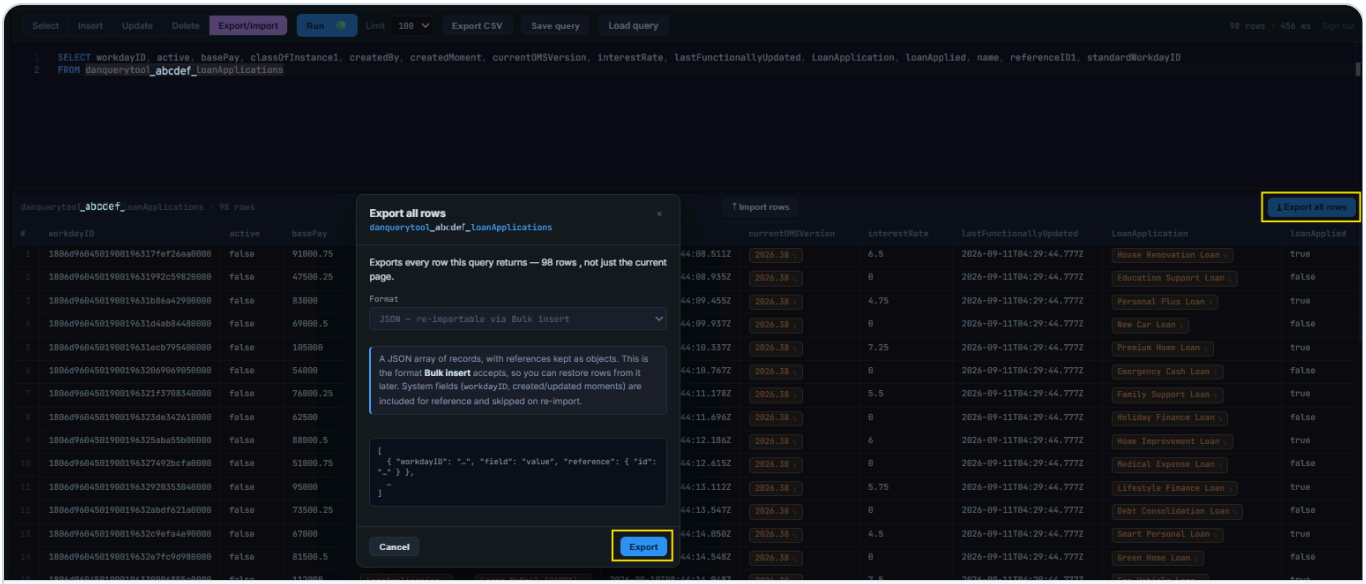
> **Custom BOs only**, like the write modes. Switch to **Export/Import** and the tree filters to objects carrying your `BO Company ID`.

Exporting

Pick a custom BO with **use**, the editor is filled with a `SELECT` of every field on that object, so nothing is left out of the backup. Run it, then choose ↓ **Export all rows**.



In Export/Import mode, 'use' on a custom BO builds a SELECT of all its fields.



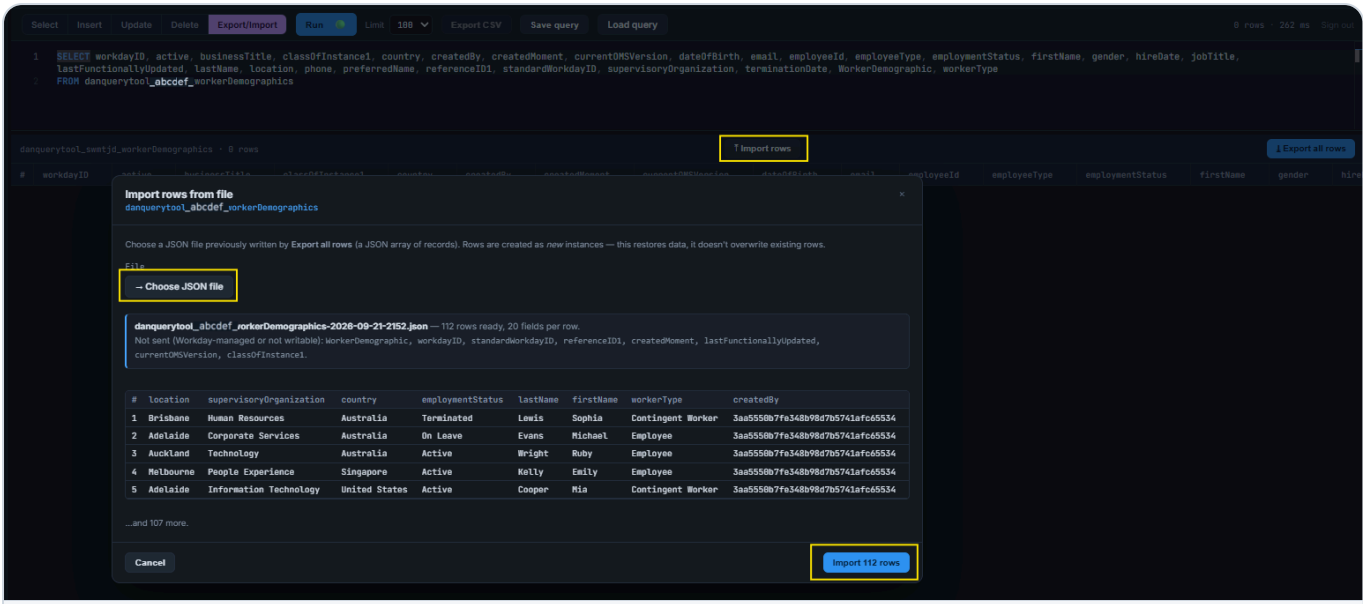
Export all rows, the whole object (here 98 rows), not just the current page.

Format	Use it for
JSON	Backups you may restore. A JSON array of records with references kept as objects, this is the format **Import rows** reads.
CSV	Reading in Excel. Flat, with references shown by descriptor, *not* re-importable.

The export pages through *every row your query matches* regardless of the toolbar's Limit, shows a running count as it goes, and finishes with a **Save As** dialog so you choose the name and folder. Files are named `<boName>-<timestamp>.json` by default. If your query has a `WHERE`, only the matching rows are exported.

Importing

Choose ⤒ **Import rows** and pick a JSON file written by Export. Before anything is sent you'll see how many rows were found, a preview of the first few, and which fields are being left out. **Import** then creates them in batches of 100, reporting each row's outcome and new id.



Import rows, the file's rows are previewed, with skipped fields listed, before anything is sent.

Fields Workday manages are never sent, because they'd be rejected: `workdayID`, `createdMoment`, `lastFunctionallyUpdated`, `currentOMSVersion`, `classOfInstance1`, `referenceID1`, and the object's own *self-reference* field, a field named after the BO whose id is the row's own `workdayID`. References are reduced to their `id`, and values the schema says are numeric or boolean are converted back from the text WQL returns.

Imported rows are new instances. Workday assigns a fresh `workdayID` on create, so an export → delete → import cycle restores the *data* but not the original ids, anything that referenced the old instances won't point at the new ones.

Limits. Import reads up to 2000 rows per file. If Workday rejects a row, its message appears in the result list, which scrolls sideways for long errors.

Backup & restore walkthrough

An end-to-end example: clear out a set of test records safely, keeping a copy you can load back.

1. Find the rows

Switch to **Delete** mode, pick your custom BO, and write a query for the rows you want gone, for example:

```
SELECT workdayID, applicantName
FROM acmequerytool_abcdef_loanApplications
WHERE applicantName contains 'TEST'
```

Run it and check the match count on **Delete all N**.

2. Export them to JSON

Click **Delete all N**, then ↓ **Go to Export (all fields)…** in the confirmation. The app switches to Export/Import mode and loads the same query with every field of the BO selected. Run it, click ↓ **Export all rows**, keep **JSON**, and save the file.

> **Why all fields?** An export only contains the columns your query selects. Going through *Go to Export* guarantees the backup holds every field, not just the two in the query above.

3. Delete

Switch back to **Delete** mode, run the query again, click **Delete all N**, tick *"I've exported a backup"*, and click **Delete all**. Check the per-row results.

4. Restore (if needed)

In **Export/Import** mode, choose ↑ **Import rows** and pick the JSON file. Review the preview and skipped fields, then **Import**.

> **Restored rows get new ids.** The data comes back, but each record has a new `workdayID`. Anything that referenced the old instances, including references from other BOs, won't point at the new ones.

Keyboard & tips

- `Ctrl` + `Space`, trigger autocomplete
- `Ctrl` + `/`, toggle a line comment (editor-only; stripped before the query runs)

Reference filters use `in (…)`. Drilling and instance filters build `WHERE field in (<id>)`; `=` is only used for boolean / date / numeric / text values.

Where things are stored. Connections and pane sizes live in your browser (localStorage). Saved queries are files you keep yourself.

Limits at a glance

What	Limit	Notes
Rows per page in the grid	100	Toolbar Limit: 10 / 25 / 50 / 100. 100 is the API's maximum page size.
Fields loaded per source in the tree	2,000 at first	Use *load all* for the rest.
Export CSV (toolbar)	100,000 rows	Pages the whole result set.
Bulk insert / Import batch	100 rows per call	Larger inputs are split into batches automatically.
Import file	2,000 rows	JSON exports only.
Bulk edit	1,000 rows	Sent in batches of 100.
Delete (selected or all)	1,000 rows	Sent in batches of 100.
Settings file import	5 MB	Connections JSON.

Troubleshooting

You see	What it means / what to do
You're sent back to the Connect screen	Your session expired (Workday access tokens are short-lived). Click **Connect** again, your query text is lost, so **Save query** anything long first.
Login fails, or the browser can't reach `localhost:3000`	Another program is using port 3000, or the API client's Redirect URI isn't exactly `http://localhost:3000/api/auth/callback`.
A business object or field isn't listed, or a query returns nothing you expect	Usually the API client's **scopes**. Add the functional area on the Developer Site and reconnect.
"Set a BO Company ID on this connection to use Insert, Update or Delete mode."	Edit the connection and fill in `BO Company ID`.
"... mode works only on custom BOs (those containing "_abcdef_")."	The query's `FROM` isn't one of your custom BOs. Pick one from the tree (it filters to custom BOs in write modes).
A **WQL error** box instead of results	Workday rejected the query; the message says why. Check clause order (see the cookbook) and that instance filters use `in (…)`, not `=`.
Bulk edit or Delete all shows *"... capped at 1,000. Add a narrower WHERE."*	Too many rows match. Split the job with a tighter `WHERE` and repeat.
An insert quietly leaves some fields out	Workday rejected them as read-only or derived (error `A13`); the app dropped them and retried. Expected for fields like `createdMoment`.
Some rows fail in a bulk result	Each failed row shows Workday's message; the others still went through. Fix and resend just the failed rows.

WQL cookbook

WQL reads a lot like SQL, with a few Workday-specific twists. These are the patterns you'll reach for most, the schema tree and autocomplete fill in the field and source names.

Select, sort, limit

```
SELECT worker, fullName, location
FROM allWorkers
ORDER BY fullName ASC
LIMIT 50
```

Rename a column with `AS`, e.g. `SELECT fullName AS "Worker Name"`. Note `LIMIT` (a query clause) caps the whole result; the toolbar's **Limit** is separate and just pages what's already returned.

Filter text

```
WHERE fullName contains 'Maria'
WHERE lastName startswith 'N'
WHERE email endswith '@example.com'
WHERE firstName in ('John', 'Jane')
```

`contains`, `startswith` and `endswith` are case-insensitive. Invert with `not contains` or `not in`.

Filter numbers & dates

```
WHERE age > 50
WHERE yearsOfService >= 5 AND yearsOfService < 10
WHERE hireDate >= '2024-01-01' AND hireDate <= '2024-12-31'
```

There's no `BETWEEN`, use two conditions. Dates are `YYYY-MM-DD`; datetimes `YYYY-MM-DD HH:MM:SS` (append `Z` for UTC).

Empty or not

```
WHERE terminationDate is empty
WHERE email is not empty
```

These back the column menu's *Is blank* / *Is not blank*.

References & instances

Instance and reference fields filter with `in (<id>)`, never `=`, exactly what clicking (or Ctrl-clicking) a reference chip builds:

```
WHERE manager_Level01 in (1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d)
WHERE location in (2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e,
                   3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f)
```

Related fields (RBO)

Pull fields from a related business object with `{ }` in the SELECT, and filter them with `WHERE ON`, which must come *before* a plain `WHERE`:

```
SELECT worker, dependents{legalName_LastName, age}
FROM allWorkers
WHERE ON dependents age > 18
WHERE location in (2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7e)
```

Prompted data sources

Some sources take a filter and/or prompt values in the `FROM` clause, the tree's `⚙ Filters & prompts` entry builds these for you. Built-in prompts can also go up front with `PARAMETERS`:

```
PARAMETERS reportingDate = "2018-01-01"
SELECT contractRateSheet, contractRates
FROM contractRateSheetsForCompany(company = 4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a)
```

Group & count

```
SELECT location, count() AS Headcount, max(yearsOfService) AS Seniority
FROM allWorkers
GROUP BY location
HAVING count() > 100
ORDER BY Headcount DESC
```

Aggregations: `COUNT()`, `COUNT(DISTINCT field)`, `SUM`, `AVG`, `MIN`, `MAX`. You can't group by multi-instance or currency fields.

Point-in-time

Query a standard Workday source as it was on a given date:

```
SELECT worker, location
FROM allWorkers (effectiveAsOfDate = "2024-01-01")
```

Clause order is fixed: `PARAMETERS → SELECT → FROM → WHERE ON → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT`. WQL sorts alphabetically, and without `ORDER BY` the row order isn't guaranteed.